LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Quickselect and Merge-Sort Counting in Practice

Week 6 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

The four-step process (same as Week 3)

1. **First instinct.** Write the algorithm that follows most directly from the problem statement.
2. **Measure why it hurts.** Compute the real complexity and plug in the problem's actual constraints.
3. **Find the structural insight.** What lets you avoid redoing work? For Divide & Conquer, this is almost always: *recurse into only the part that can possibly contain the answer*, or *piggyback bookkeeping onto an existing recursive combine step*.
4. **Implement, trace by hand, then look for programming-level tricks.**

Today's two problems

Medium – Kth Largest Element in an Array: quickselect, this week's expected $\Theta(n)$ selection algorithm. **Hard – Count of Smaller Numbers After Self**: a direct extension of this week's *counting inversions during merge* technique.

Problem statement

LeetCode 215 – Kth Largest Element in an Array (Medium)

Given an integer array `nums` and an integer k, return the k-th largest element in the array (the k-th largest, *not* the k-th distinct element).

- Constraints (LeetCode): $1 \leq k \leq \text{nums.length} \leq 10^5$.
- "k-th largest" is the same **selection problem** from this week's lecture, just indexed from the top: the k-th largest is the element at ascending index $n - k$ (0-indexed) – i.e. `Quickselect(A, n-k)` in the lecture's notation.

Worked example

`nums = [3,2,1,5,6,4]`, `k = 2` $\Rightarrow$ expected output 5.

- Sorted ascending: $[1, 2, 3, 4, 5, 6]$. The "2nd largest" sits at ascending index $n - k = 6 - 2 = 4$, and $[1, 2, 3, 4, \mathbf{5}, 6][4] = 5$. ✓
- Three ways to get there:
  1. Repeatedly scan for the current maximum, remove it, repeat k times.
  2. Sort the whole array, then index.
  3. **Quickselect**: partition around a pivot, recurse into *only* the side that must contain index $n - k$.

First instinct: repeatedly extract the maximum

Most direct reading of "k-th largest"

Find the maximum, remove it; find the new maximum, remove it; ... after k extractions, the k-th one found is the answer.

1: **for** $t \leftarrow 1$ **to** k **do**
2: scan all remaining elements, find and remove the maximum
3: **end for**
4: **return** the last element removed

Each scan is $\Theta(n)$, repeated k times: $\Theta(nk)$.

Why this is bad

- $\Theta(nk)$ – fine for small k, but at $k = n/2$ (a valid input) this is $\Theta(n^2)$: at $n = 10^5$, roughly 5×10^9 operations.
- Sorting the whole array first is $\Theta(n \log n)$, always better than the naive loop for large k – but it fully **orders every element**, when the problem only asks for *one* order statistic.

The smell to notice

Whenever you only need *one* piece of information about a total order (the minimum, the median, the k-th value) but the algorithm computes the *entire* order first, ask: can I discard the part of the data that is provably irrelevant, the way binary search discards half the array without looking at it?

The structural insight

- After partitioning around a pivot p into *less*, *equal*, *greater*, the target index k' falls into *exactly one* of the three groups – and once you know which, the other two groups are provably irrelevant to the answer and can be thrown away entirely.
- This is this week's QUICKSELECT (lecture pseudocode), applied with target index $k' = n - k$ for "k-th largest."
- With a *uniformly random* pivot, the lecture proves $T(n) \leq T(3n/4) + O(n)$ in expectation – a decreasing geometric series, so $T(n) = \Theta(n)$ *expected*, even though a single sort already needs $\Theta(n \log n)$ *every* time.

Optimal algorithm: pseudocode

```
1: function KthLargest(A, k)
2:     return Quickselect(A, |A| − k)                ▷ k-th largest = ascending index n − k
3: end function
4: function Quickselect(A, k)
5:     choose a pivot p from A                       ▷ random pivot for expected Θ(n)
6:     less, equal, greater ← partition A by comparison to p
7:     if k < |less| then
8:         return Quickselect(less, k)
9:     else if k < |less| + |equal| then
10:        return p
11:    else
12:        return Quickselect(greater, k − |less| − |equal|)
13:    end if
14: end function
```

Expected $\Theta(n)$ with a random pivot; worst case $\Theta(n^2)$ with a *consistently* bad pivot (same theorem as this week's lecture).

Trace on the worked example

`nums = [3,2,1,5,6,4]`, target index $k' = n - k = 6 - 2 = 4$. Using the *deterministic last-element* pivot rule from the lecture's exercises:

call	array	pivot	k'	outcome
1	$[3, 2, 1, 5, 6, 4]$	4	4	*less*=$[3, 2, 1]$, *equal*=$[4]$, *greater*=$[5, 6]$; k'=4 is not $<$ $\lvert less\rvert + \lvert equal\rvert$=4 $\Rightarrow$ recurse right, $k'' = 4 - 3 - 1 = 0$
2	$[5, 6]$	6	0	*less*=$[5]$, *equal*=$[6]$; $0 <$ $\lvert less\rvert$=1 $\Rightarrow$ recurse left
3	$[5]$	–	0	single element: **return** 5

Result: 5 – matches the expected output.

Complexity: naive vs. optimal

Approach	Time	Notes
Extract max, k times	$\Theta(nk)$	$\Theta(n^2)$ when $k \approx n/2$
Sort, then index	$\Theta(n \log n)$	always orders far more than needed
Quickselect	$\mathbf{\Theta(n)}$ **expected**	$\Theta(n^2)$ worst case without randomization

Python implementation

```python
import random

def findKthLargest(nums, k):
    target = len(nums) - k          # k-th largest = ascending index n-k

    def quickselect(arr, k):
        if len(arr) == 1:
            return arr[0]
        pivot = random.choice(arr)                    # avoid adversarial worst case
        less    = [x for x in arr if x < pivot]
        equal   = [x for x in arr if x == pivot]
        greater = [x for x in arr if x > pivot]
        if k < len(less):
            return quickselect(less, k)
        elif k < len(less) + len(equal):
            return pivot
        else:
            return quickselect(greater, k - len(less) - len(equal))

    return quickselect(nums, target)
```

Implementation tips & tricks (the programming side)

- **Randomize the pivot – this is not optional on LeetCode.** The lecture notes that a deterministic "always last element" pivot is "fine for the exercises." On LeetCode it is not: judges routinely include already-sorted and reverse-sorted test cases specifically because they are the worst case for a fixed-position pivot, triggering the proven $\Theta(n^2)$ blow-up. `random.choice(arr)` converts that adversarial worst case into an astronomically unlikely one – exactly the theorem this week proves.
- **Recursion depth is a real (if rare) risk here too.** Even with randomization, an $O(n)$-deep call chain has nonzero probability; at $n = 10^5$ this can still exceed Python's recursion limit. Because each quickselect call is *tail-recursive* (the recursive call's result is returned immediately, with no work after it), it converts trivially into a `while` loop – unlike Week 3's DFS, where work happens *after* the recursive call returns and an explicit stack was needed instead.
- **The 3-list partition here is $O(n)$ extra space per level.** It matches the lecture exactly and is easy to prove correct; a production implementation would use Hoare's in-place partition scheme for $O(1)$ extra space, at the cost of a fiddlier correctness argument.

Problem statement

LeetCode 315 – Count of Smaller Numbers After Self (Hard)

Given an integer array `nums`, return an array `counts` where `counts[i]` is the number of elements to the **right** of `nums[i]` that are **strictly smaller** than `nums[i]`.

- Constraints (LeetCode): $1 \leq \text{nums.length} \leq 10^5$.
- This is **this week's "counting inversions during merge"** technique, generalized: instead of one grand total, report the count *per element*.

Worked example

`nums = [5,2,6,1]` $\Rightarrow$ expected output $[2, 1, 1, 0]$.

- Right of 5: $\{2, 6, 1\}$, smaller than 5: $\{2, 1\} \Rightarrow 2$.
- Right of 2: $\{6, 1\}$, smaller than 2: $\{1\} \Rightarrow 1$.
- Right of 6: $\{1\}$, smaller than 6: $\{1\} \Rightarrow 1$.
- Right of 1: $\{\} \Rightarrow 0$.

First instinct: check every pair

Most direct reading of the problem

For each index i, scan every index $j > i$ and count how many satisfy $\texttt{nums}[j] < \texttt{nums}[i]$.

```
1: for i ← 0 to n − 1 do
2:     counts[i] ← 0
3:     for j ← i + 1 to n − 1 do
4:         if nums[j] < nums[i] then
5:             counts[i] ← counts[i] + 1
6:         end if
7:     end for
8: end for
```

$\Theta(n^2)$ total – exactly the brute-force inversion count the lecture opens with.

Why this is bad

- $\Theta(n^2)$: at $n = 10^5$, about 10^{10} comparisons – far too slow.
- This is the *identical* shape of blow-up as plain inversion counting; the lecture already tells us the fix: **piggyback the counting onto mergesort**.

The smell to notice

"For each element, count how many later elements satisfy some comparison" is exactly the inversion-counting pattern – whenever you see it, reach for the merge-and-count technique instead of nested loops.

The structural insight

- Carry each value's **original index** alongside it through the sort: work with pairs $(\text{value}, \text{original index})$.
- Split by *position* (not value) into a left half and a right half exactly as in mergesort – every element of the right half is, by construction, still to the *right* of every element of the left half in the original array.
- During MergeAndCount, whenever the merge is about to emit a **left**-half element $L[i]$ (because $L[i] \leq R[j]$), every right-half element already emitted – there are exactly j of them – is strictly smaller than $L[i]$ *and* originally to its right. So: $counts[L[i].\text{index}] \mathrel{+}= j$.
- Recursing on the left and right halves accumulates each element's count from *within* its own half; the merge step above adds the *cross*-half count. Same recurrence as plain mergesort: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Optimal algorithm: pseudocode

```
1: function MergeAndCount(L, R, counts)                ▷ L, R: sorted (value, index) pairs
2:     M ← empty; i ← 0; j ← 0
3:     while i < |L| and j < |R| do
4:         if L[i].val ≤ R[j].val then
5:             counts[L[i].idx] += j                   ▷ j smaller right-elements already emitted
6:             append L[i] to M; i ← i + 1
7:         else
8:             append R[j] to M; j ← j + 1
9:         end if
10:    end while
11:    append remaining elements of L (each gets counts += j = |R|) and of R
12:    return M
13: end function
```

Wrapped in an ordinary mergesort over (value, index) pairs: $\Theta(n \log n)$ total.

Trace on the worked example

`nums = [5,2,6,1]`, pairs $(5,0),(2,1),(6,2),(1,3)$. Split into left $= \{(5,0),(2,1)\}$, right $= \{(6,2),(1,3)\}$.

step	merge	effect on `counts`
1	merge $[(5,0)]$, $[(2,1)]$: emit $(2,1)$ first (j:0→1), then $(5,0)$	j=1 when $(5,0)$ emitted: *counts*$[0] \mathrel{+}= 1$
2	merge $[(6,2)]$, $[(1,3)]$: emit $(1,3)$ first (j:0→1), then $(6,2)$	j=1 when $(6,2)$ emitted: *counts*$[2] \mathrel{+}= 1$
3	merge $[(2,1),(5,0)]$, $[(1,3),(6,2)]$: emit $(1,3)$; emit $(2,1)$ at j=1; emit $(5,0)$ at j=1; emit $(6,2)$	*counts*$[1] \mathrel{+}= 1$; *counts*$[0] \mathrel{+}= 1$ (total 2); final: $[2,1,1,0]$

Result: $[2,1,1,0]$ – matches the expected output exactly.

Complexity: naive vs. optimal

Approach	Time	At $n = 10^5$
All pairs, nested loops	$\Theta(n^2)$	$\sim 10^{10}$ comparisons
Merge-sort with index tracking	$\boldsymbol{\Theta(n \log n)}$	$\sim 1.7 \times 10^6$ operations

Roughly four orders of magnitude apart, from re-using a combine step already being computed for free.

Python implementation (part 1: merge with counting)

```python
def countSmaller(nums):
    n = len(nums)
    counts = [0] * n
    indexed = list(enumerate(nums))      # [(original_index, value), ...]

    def merge_and_count(pairs):
        if len(pairs) <= 1:
            return pairs
        mid = len(pairs) // 2
        left = merge_and_count(pairs[:mid])
        right = merge_and_count(pairs[mid:])
        merged, i, j = [], 0, 0
        while i < len(left) and j < len(right):
            if left[i][1] <= right[j][1]:
                counts[left[i][0]] += j         # j smaller right-elements seen so far
                merged.append(left[i]); i += 1
            else:
                merged.append(right[j]); j += 1
```

Python implementation (part 2: leftovers & driver)

```
        while i < len(left):                    # leftovers: all of ‘right‘ was
            smaller
            counts[left[i][0]] += len(right) - j
            merged.append(left[i]); i += 1
        merged.extend(right[j:])
        return merged

    merge_and_count(indexed)
    return counts
```

Note the leftover-handling line: once `right` is exhausted first, every remaining `left` element has *all* of `right` (size `len(right) - j` at that point, but since `j` no longer advances, it is simply `len(right)` for each) to its right and smaller – the same “append the tail” step from the lecture’s plain MERGE, just with a matching count update.

Implementation tips & tricks (the programming side)

- **Carry (index, value) pairs, never bare values.** The entire algorithm depends on knowing *where* each value originally came from once it has been reordered by the sort – losing the index is the single most common bug when adapting mergesort to this problem.
- `pairs[:mid]` / `pairs[mid:]` **slicing is $O(n)$ per call.** It does not change the asymptotic bound (still $\Theta(n \log n)$ overall, same as ordinary Python mergesort), but a maximally tight implementation passes index ranges into a shared array instead of allocating new lists at every level, roughly halving constant-factor overhead in Python.
- **Recursion depth is $\Theta(\log n)$ here, not $\Theta(n)$** – unlike quickselect's worst case, mergesort's recursion always halves regardless of the data, so at $n = 10^5$ the depth is only ≈ 17: no recursion-limit concerns, in contrast to both problems in Week 3's deck and quickselect above.

Summary: two problems, one lesson

1. **Kth Largest Element** needs only *one* order statistic, so quickselect recurses into a single shrinking partition – expected $\Theta(n)$, strictly less work than fully sorting.
2. **Count of Smaller Numbers After Self** needs a count *per element*, so it reuses mergesort's already-happening combine step and attaches one extra piece of bookkeeping – $\Theta(n \log n)$, the same bound as sorting itself, for free.
3. Both times, the naive instinct re-derived information from scratch (rescanning for each extraction, or comparing every pair); both times, the fix was to recognize that a *recursive combine step already being computed* could carry the answer along with it, at no extra asymptotic cost.
4. The randomization trick in quickselect and the index-tracking trick in merge-and-count are both, in their own way, about turning a *worst case* (bad pivots; losing track of position) into something the algorithm is structurally protected against.

Keep practicing

`LEETCODE.md` lists more Week 6 problems (*Sort an Array*, *Maximum Subarray*, *Pow(x, n)*, *Reverse Pairs*, *Different Ways to Add Parentheses*, ...) – try the same four-step process on each.